



資料分析與機器學習 (Data Analysis and Machine Learning)

“數據和特徵決定了機器學習的上限，
而模型和演算法只是逼近這個上限而已”

Lecture 5 –
Data Preprocessing & Scikit-learn

教師：陳慶永

Outline

- Data Preprocessing (資料前處理)
 - ▶ Partitioning a dataset into separate training and test datasets
 - ▶ Dealing with missing data (缺失資料)
 - ▶ Handling categorical data (類別型資料)
 - ▶ Feature scaling (特徵縮放)
 - ▶ Feature selection (特徵選取) and feature extraction (特徵提取)
- Regularization (正則化)
- Scikit-learn
- Training a Classifier with Scikit-learn
 - ▶ Pipeline (管線) based Workflows

Discussed in a later lecture

將實作提升到管線化
流程的層次



Data Preprocessing

Data is the King

- The available **data** in our training dataset is even more important than the choice of an appropriate learning algorithm
 - ▶ The quality of the data and the amount of useful information it contains are key factors that determine how well an ML algorithm can learn
 - No algorithm will be able to make good predictions without informative and discriminatory features
 - ▶ We have to examine and **preprocess** a dataset before feeding it to an ML algorithm
 - Removing and imputing missing values
 - Getting categorical data into shape
 - Selecting relevant features

Data Preprocessing

- Raw Data (原始數據)

Split dataset first: Training and Test datasets

- Data Cleaning

- ▶ 資料清理，刪/補空值、剔異常

- Data Preprocessing – Encoding

- ▶ 資料編碼，把類別型資料轉成數值

- Feature Engineering – Feature Generation

- ▶ 衍生新特徵，取絕對值、特徵相乘、離散化

- Data Preprocessing – Feature Scaling

- ▶ 特徵縮放，標準化、正規化

- Feature Engineering – Dimensionality Reduction

- ▶ 降維處理，特徵選取、特徵提(萃)取

- Model Training (模型訓練)

廣義的資料前處理

Partitioning a Dataset

- First of all, splitting your data into *two separate subsets*
 - ▶ **Training set** to build the model
 - ▶ **Test set** to evaluate its performance on *unseen* data
- The Golden Rule: Split *Before* Processing
 - ▶ To prevent data leakage (unintentionally exposing information from the test set to the model during training)
 - ▶ *Only* the training set is used to *fit* the models (such as estimators and transformers)
 - ▶ *Apply* those *fitted* models to the test dataset

Partitioning a Dataset

- Choosing an *appropriate ratio* for partitioning a dataset into training and test datasets
 - ▶ The larger the test set, the less valuable information that the learning algorithm could benefit from
 - ▶ The smaller the test set, the more inaccurate the estimation of the generalization (泛化) error
 - ▶ It's all about *balancing* this tradeoff
- Most commonly used splits are 60:40, **70:30**, or 80:20, depending on the size of the initial dataset

Missing Data

- In real-world applications, one or more values in the training examples may be *missing* for various reasons
 - ▶ Error in the data collection process
 - ▶ Certain measurements may not be applicable
 - ▶ Particular fields could have been simply left blank in a survey
- Most computational tools are unable to handle such missing values
 - ▶ **Blank spaces**, **NaN** (not a number), **NULL** (unknown values in databases)

Missing Data

■ Example: DataFrame with missing values

```
>>> import pandas as pd
>>> from io import StringIO
>>> csv_data = \
...     '''A,B,C,D
...     1.0,2.0,3.0,4.0
...     5.0,6.0,,8.0
...     10.0,11.0,12.0,'''
>>> # If you are using Python 2.7, you need
>>> # to convert the string to unicode:
>>> # csv_data = unicode(csv_data)
>>> df = pd.read_csv(StringIO(csv_data))
>>> df
```

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN

Detect Missing Values

- For a larger DataFrame, it can be tedious to look for missing values manually
 - ▶ Use the `isnull()` method to return a DataFrame with Boolean values
 - True: a missing value (ex: NaN, None, NaT)
 - False: valid data (including “ ” 空白字串)
 - Usually followed by the `sum()` method to count the number of missing values per column

使用 `replace()` 方法搭配正規表達式將空白字串轉換成 `np.nan` :

```
df = df.replace(r'^\s*$', np.nan, regex=True)
```

```
>>> df.isnull().sum()
A      0
B      0
C      1
D      1
```

Dealing with Missing Values

■ Techniques for dealing with missing values

▶ **Removing** *entries* from the dataset

- Convenient but... not feasible

- May remove too many samples (making a reliable analysis impossible)
- May remove too many feature columns (losing valuable information that our classifier needs)

▶ **Imputing** *missing values* from other training examples and features

- Interpolation techniques

Eliminating Training Samples or Features

- Use the `dropna()` method to remove the corresponding training examples (rows) or features (columns) from the dataset entirely

```
>>> df.dropna(axis=0)
```

	A	B	C	D
0	1.0	2.0	3.0	4.0

```
>>> df.dropna(axis=1)
```

	A	B
0	1.0	2.0
1	5.0	6.0
2	10.0	11.0

Eliminating Training Samples or Features

■ Usage of additional parameters of dropna()

```
>>> # only drop rows where all columns are NaN
>>> # (returns the whole array here since we don't
>>> # have a row with all values NaN)
>>> df.dropna(how='all')
```

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN

```
>>> # drop rows that have fewer than 4 real values
```

```
>>> df.dropna(thresh=4)
```

	A	B	C	D
0	1.0	2.0	3.0	4.0

```
>>> # only drop rows where NaN appear in specific columns (here: 'C')
```

```
>>> df.dropna(subset=['C'])
```

	A	B	C	D
0	1.0	2.0	3.0	4.0
2	10.0	11.0	12.0	NaN

Imputing Missing Values

- Use different *interpolation* techniques to *estimate* the missing values from other training examples

- ▶ Mean imputation

- Replace the missing value with the *mean value* of the entire feature column

- ▶ Most-frequent imputation

- Replace missing using the *most frequent value* of the entire feature column (尤其適合類別型特徵)

- ▶ KNN imputation

- Imputation for completing missing values using *k-Nearest Neighbors* (e.g., average)

Handling Categorical Data

- Real-world datasets may contain one or more *categorical* feature columns
 - ▶ Ordinal (順序型)
 - Can be sorted or ordered
 - Ex: T-shirt size (XL > L > M)
 - ▶ Nominal (名義型)
 - Don't imply any order
 - Ex: T-shirt color (red, green, blue, etc.)
- Categorical data has to be *mapped to numerical values* for computing with a machine

Handling Categorical Data

■ Example: DataFrame with Categorical Data

```
>>> import pandas as pd
>>> df = pd.DataFrame([
...     ['green', 'M', 10.1, 'class2'],
...     ['red', 'L', 13.5, 'class1'],
...     ['blue', 'XL', 15.3, 'class2']])
>>> df.columns = ['color', 'size', 'price', 'classlabel']
>>> df
```

	color	size	price	classlabel
0	green	M	10.1	class2
1	red	L	13.5	class1
2	blue	XL	15.3	class2

Nominal

Ordinal

Numerical

Nominal

Note: We do not use ordinal information in class labels.

Mapping Ordinal Features

- Convert the categorical string values into **integers**
 - ▶ Define the **mapping** manually based on the knowledge about the order

```
>>> size_mapping = {'XL': 3,  
...                 'L': 2,  
...                 'M': 1}
```

```
>>> df['size'] = df['size'].map(size_mapping)  
>>> df
```

	color	size	price	classlabel
0	green	1	10.1	class2
1	red	2	13.5	class1
2	blue	3	15.3	class2

- ▶ Reverse-mapping

```
>>> inv_size_mapping = {v: k for k, v in size_mapping.items()}  
>>> df['size'].map(inv_size_mapping)
```

0	M
1	L
2	XL

Encoding Class Labels

- Many ML libraries require that class labels are encoded as **integers**
- Class labels are *not* ordinal
 - ▶ It doesn't matter which integer number we assign to a particular string label

```
>>> import numpy as np
>>> class_mapping = {label: idx for idx, label in
...                  enumerate(np.unique(df['classlabel']))}
>>> class_mapping
{'class1': 0, 'class2': 1}
```

Encoding Class Labels

■ Mapping

```
>>> df['classlabel'] = df['classlabel'].map(class_mapping)
>>> df
```

	color	size	price	classlabel
0	green	1	10.1	1
1	red	2	13.5	0
2	blue	3	15.3	1

■ Reverse-mapping

```
>>> inv_class_mapping = {v: k for k, v in class_mapping.items()}
>>> df['classlabel'] = df['classlabel'].map(inv_class_mapping)
>>> df
```

	color	size	price	classlabel
0	green	1	10.1	class2
1	red	2	13.5	class1
2	blue	3	15.3	class2

One-hot Encoding on Nominal Features

■ Common mistakes

```
blue = 0  
green = 1  
red = 2
```

- ▶ Model will assume that $green > blue$ and $red > green$

■ One-hot encoding (獨熱編碼)

- ▶ Create a new **dummy feature** for each *unique value* in the nominal feature column
- ▶ *Binary* values are used to indicate a particular value of a sample
 - Example: New features: blue, green, red
 - A blue example: encoded as blue=1, green=0, red=0
 - A green example: encoded as blue=0, green=1, red=0
 - A red example: encoded as blue=0, green=0, red=1

One-hot Encoding on Nominal Features

- A convenient way to create dummy features via one-hot encoding is the `get_dummies()` function in `pandas`
 - ▶ It only converts *string* columns (leaving all other columns unchanged)

```
>>> pd.get_dummies(df[['price', 'color', 'size']])
```

	price	size	color_blue	color_green	color_red
0	10.1	1	0	1	0
1	13.5	2	0	0	1
2	15.3	3	1	0	0

One-hot Encoding on Nominal Features

- One-hot encoding introduces multi-collinearity
 - ▶ An issue for certain methods (e.g., matrix inversion based)
 - ▶ If features are highly correlated, matrices are computationally difficult to invert (ill-conditioned)

```
>>> pd.get_dummies(df[['price', 'color', 'size']])
```

	price	size	color_blue	color_green	color_red
0	10.1	1	0	1	0
1	13.5	2	0	0	1
2	15.3	3	1	0	0



One-hot Encoding on Nominal Features

- To reduce the correlation among variables, simply **remove one feature column** from the one-hot encoded array
 - ▶ Note that we do *not* lose any important information by removing a feature column
 - If the column color_blue removed, then color_green=0 and color_red=0 implies that the observation must be blue

```
>>> pd.get_dummies(df[['price', 'color', 'size']],  
...               drop_first=True)
```

	price	size	color_green	color_red
0	10.1	1	1	0
1	13.5	2	0	1
2	15.3	3	0	0

Drop the first column

Feature Scaling

- Feature scaling is a crucial step in preprocessing that can easily be forgotten

- ▶ Standardization (see Lecture 3 Slides, pp. 21-24)

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

- ▶ **Normalization** (正規化)

- Scaling to a range of **[0, 1]** (a special case of **min-max scaling**)

$$x_{norm}^{(i)} = \frac{x^{(i)} - x_{min}}{x_{max} - x_{min}}$$

- x_{min} : The smallest value in a feature column
- x_{max} : The largest value in a feature column

注意：One-hot encoding 後產生的 0 與 1 特徵欄位不需要進行特徵縮放

Feature Scaling

Input	Standardized	Min-max normalized
0.0	-1.46385	0.0
1.0	-0.87831	0.2
2.0	-0.29277	0.4
3.0	0.29277	0.6
4.0	0.87831	0.8
5.0	1.46385	1.0

Feature Scaling

- Normalization is useful when we need values in a *bounded* interval
 - ▶ However, a single extreme *outlier* can crush the rest of the data into a very tiny sub-range
- Standardization may be more practical for many ML algorithms (especially for GD/SGD optimization)
 - ▶ Maintains useful information about outliers and makes the algorithm less sensitive to them

Feature Selection (特徴選取)

- Two main categories of dimensionality reduction techniques
 - ▶ Feature selection
 - *Select* a subset of the original features
 - ▶ Feature extraction
 - *Derive* information from the feature set to construct a new feature subspace

Sequential Feature Selection

- Sequential feature selection algorithms are a family of *greedy search* algorithms
 - ▶ Reduce an initial d -dimensional feature space to a k -dimensional feature subspace where $k < d$
 - Sequential *forward* selection (SFS) (加法概念)
 - Sequential *backward* selection (SBS) (減法概念)

Greedy algorithms make *locally* optimal choices at each stage of a combinatorial search problem and generally yield a **suboptimal** solution, in contrast to exhaustive search algorithms which evaluate *all* possible combinations and find the optimal solution.

Sequential Feature Selection

■ SBS algorithm

► Define a criterion function J

1. Initialize the algorithm with $k = d$, where d is the dimensionality of the full feature space, X_d .
2. Determine the feature, x^- , that maximizes the criterion: $x^- = \operatorname{argmax} J(X_k - x)$, where $x \in X_k$.
3. Remove the feature, x^- , from the feature set: $X_{k-1} = X_k - x^-$; $k = k - 1$.
4. Terminate if k equals the number of desired features; otherwise, go to *step 2*.



Regularization

Overfitting (過擬合)

- The model captures the patterns in the training data well but *fails to generalize* well to *unseen* data
 - ▶ The model has a **high variance**
 - Variance measures the consistency
 - ▶ Caused by having too many parameters (model too complex)

Overfitting (過擬合)

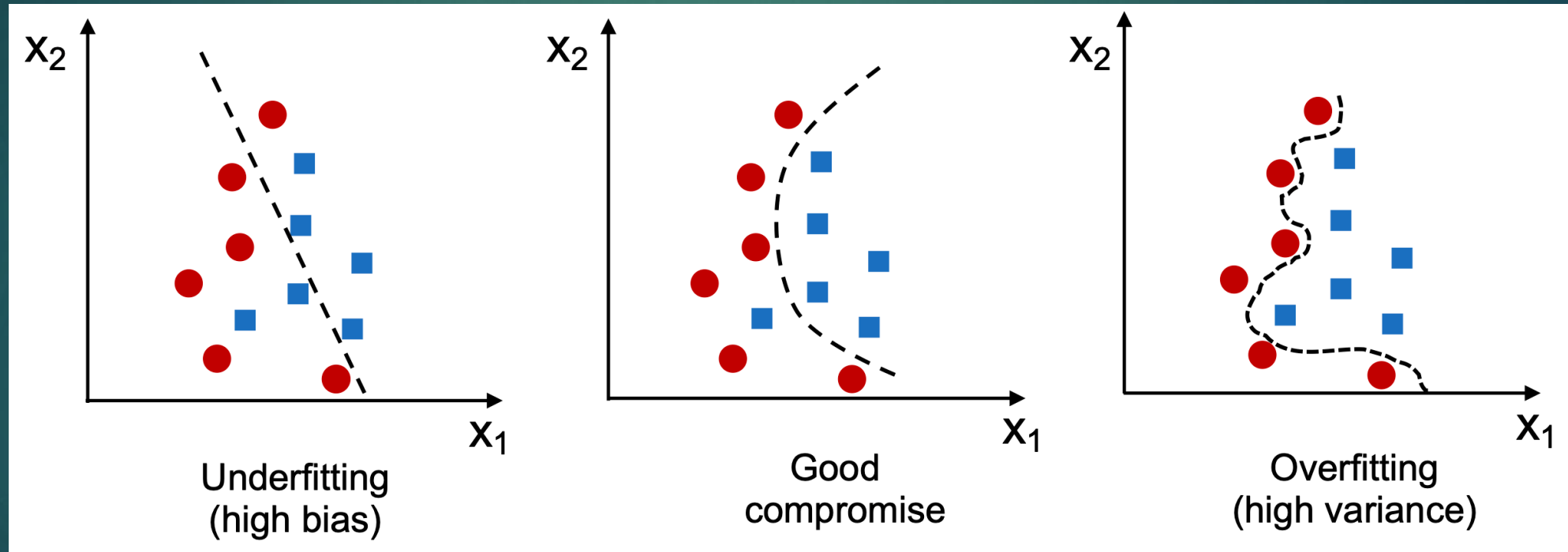
- Common solutions to prevent overfitting
 - ▶ Collect **more training data**
 - Often not applicable
 - ▶ Choose a **simpler model** (with fewer parameters)
 - ▶ Introduce a penalty for complexity via **regularization**
 - Also leads to simpler models
 - ▶ **Reduce the dimensionality** of the data
 - Also leads to simpler models

Underfitting (欠擬合)

- The model is not complex enough to capture the pattern in the training data well and therefore also suffers from low performance on unseen data
 - ▶ The model has a high bias
 - Bias measures the systematic error

The Bias-Variance Tradeoff

- Examples of underfitted, well-fitted, and overfitted models



Regularization (正則化)

- A very useful method for
 - ▶ handling collinearity (high correlation among features)
 - ▶ filtering out noise from data
 - ▶ preventing overfitting
- Concept: Introduce additional term to loss function to *penalize* extreme parameter (weight) values
 - ▶ L2 regularization (also called L2 shrinkage or weight decay)
 - ▶ L1 regularization

L2 Regularization

■ Regularization term (正則化項)

$$\frac{\lambda}{2n} \|\mathbf{w}\|^2 = \frac{\lambda}{2n} \sum_{j=1}^m w_j^2$$

- ▶ λ : Regularization parameter
 - Control the **regularization strength** (overfitting $\leftrightarrow$ underfitting)
- ▶ n : sample size
 - Scale the regularization term similar to the loss

For regularization to work properly, we need to ensure that all features are on comparable scales.

L2 Regularization

■ Example: Logistic Regression with L2 Regularization

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n [-y^{(i)} \log(\sigma(z^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(z^{(i)}))] + \frac{\lambda}{2n} \|\mathbf{w}\|^2$$

- ▶ Regularization term will *shrink* the weights during model training
- ▶ Partial derivative of the regularized loss function

$$\frac{\partial L(\mathbf{w}, b)}{\partial w_j} = \left(\frac{1}{n} \sum_{i=1}^n \underbrace{(\sigma(\mathbf{w}^T \mathbf{x}^{(i)}) - y^{(i)})}_{a^{(i)}} x_j^{(i)} \right) + \frac{\lambda}{n} w_j$$

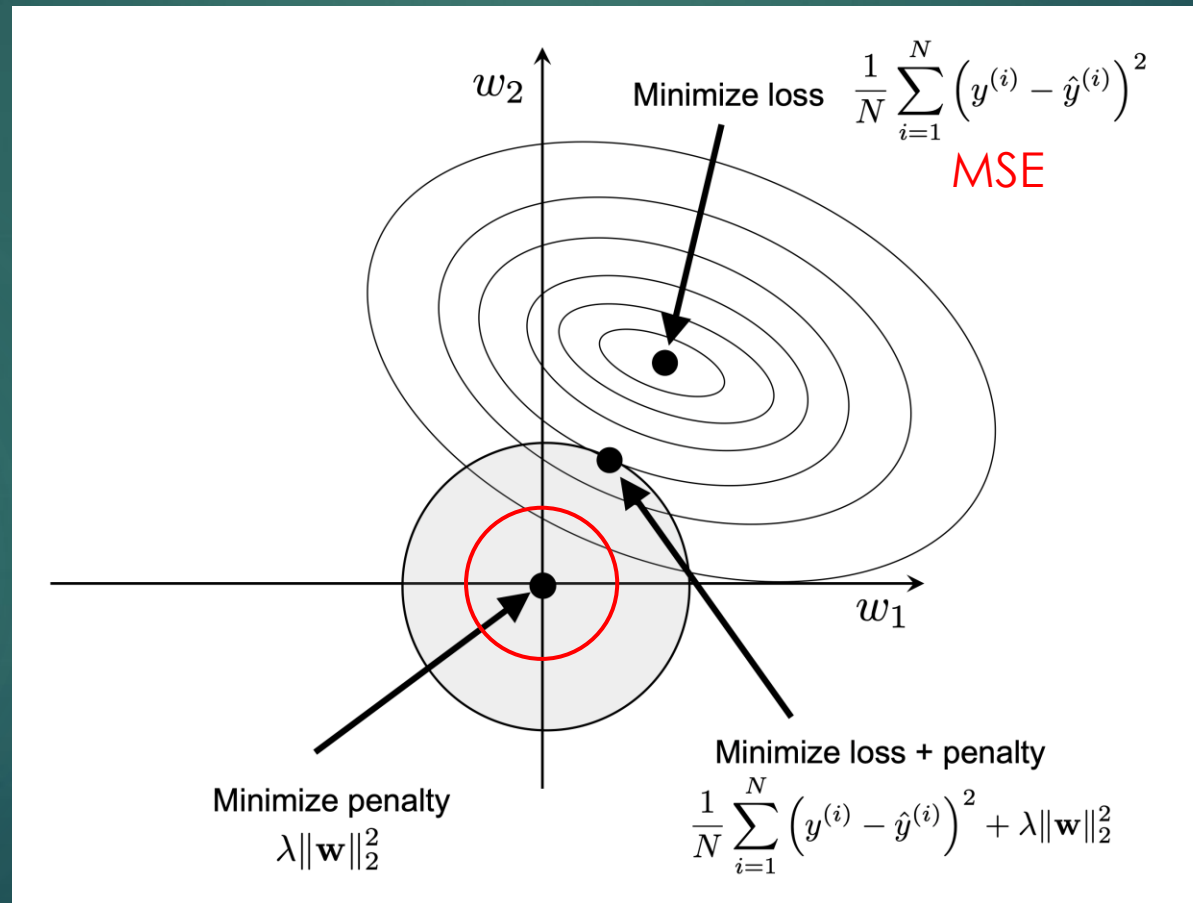
Weight update: Take steps in the *opposite* direction of the gradient

(review Lecture 4 slides, p. 19)

Note: The *bias unit* (negative threshold) is usually *not* regularized.

L2 Regularization

■ Geometric interpretation



L1 Regularization

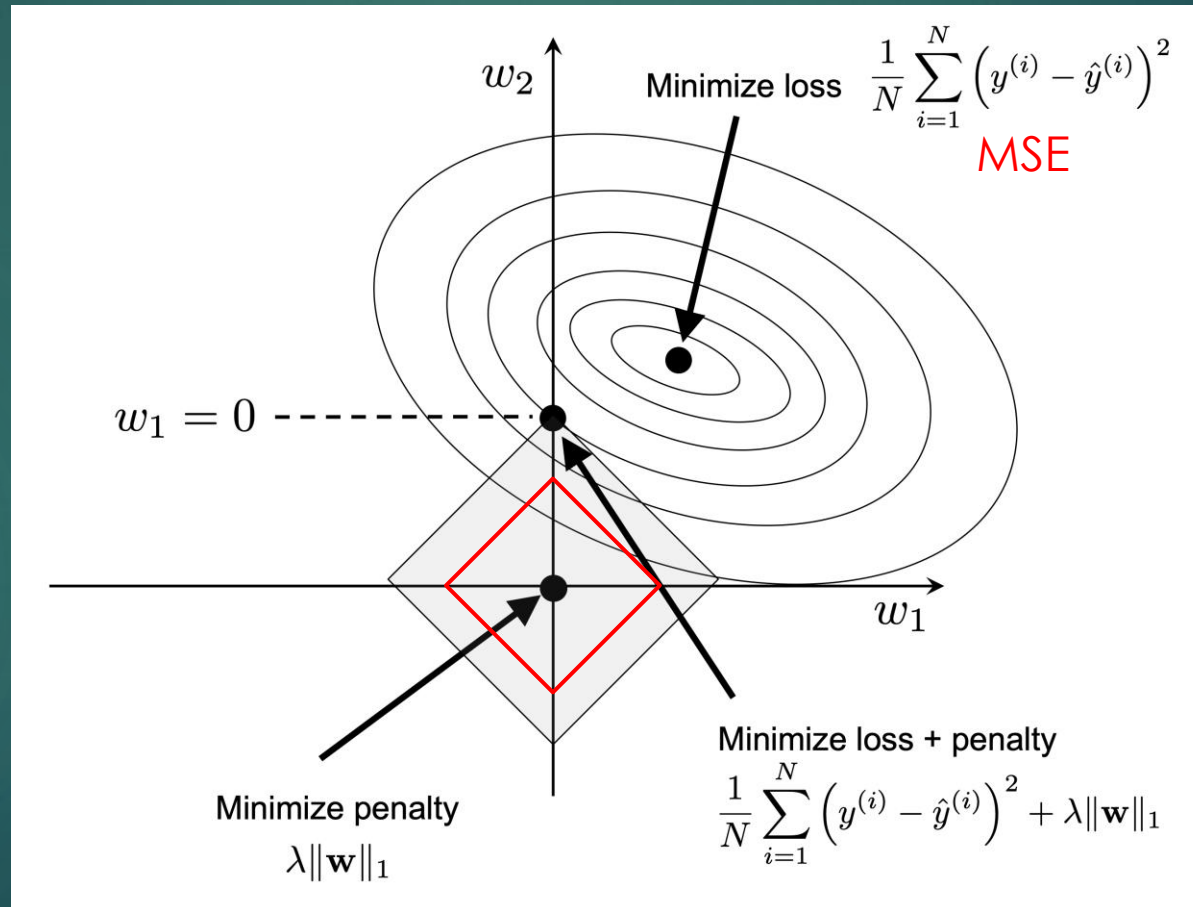
■ Regularization term

$$\lambda \|\mathbf{w}\|_1 = \sum_{j=1}^m |w_j|$$

- ▶ Usually yields *sparse* feature vectors, and most feature weights will be **zero**
- ▶ Can be treated as a technique of **feature selection**

L1 Regularization

■ Geometric interpretation





Scikit-learn

Scikit-learn

- **Scikit-learn** (**sklearn**) is an open source ML library in Python
 - ▶ Supports supervised and unsupervised learning via a *consistent* API
 - ▶ Provides various tools for data preprocessing, model fitting, model selection and model evaluation
- Main features
 - ▶ Preprocessing, ML algorithms (highly optimized implementation), grid search & cross validation, performance metrics
 - ▶ **Pipeline** (管線)
- How to use? 🤔
 - ▶ [Getting Started](#)
 - ▶ [User Guide](#)
 - ▶ [API Reference](#)

General Modules

- datasets
- preprocessing, impute, compose
- pipeline, feature_selection, decomposition
- model_selection
- metrics

Scikit-learn functions and classes often have parameters with default values that we *may* omit for clarity. You can read more about those parameters by going through the online documentation.

sklearn.datasets

- Utilities to load popular datasets and artificial data generators

▶ Example: `load_iris()` function

- Load and return the iris dataset

```
from sklearn import datasets
import numpy as np
→ iris = datasets.load_iris()
X = iris.data[:, [2, 3]]
y = iris.target
```

{0,1,2}

petal length, petal width

(review Lecture 1 slides,
pp. 55-56)

sklearn.model_selection

- Tools for model selection, such as cross validation and hyper-parameter tuning

▶ Example: `train_test_split()` function

- *Randomly split (i.e., shuffling before splitting) the dataset into separate training and test datasets*

```
from sklearn.model_selection import train_test_split
→ X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1, stratify=y
)
```

70% for training
30% for testing

Training and test subsets that have the same proportions of class labels as the input dataset

sklearn.model_selection

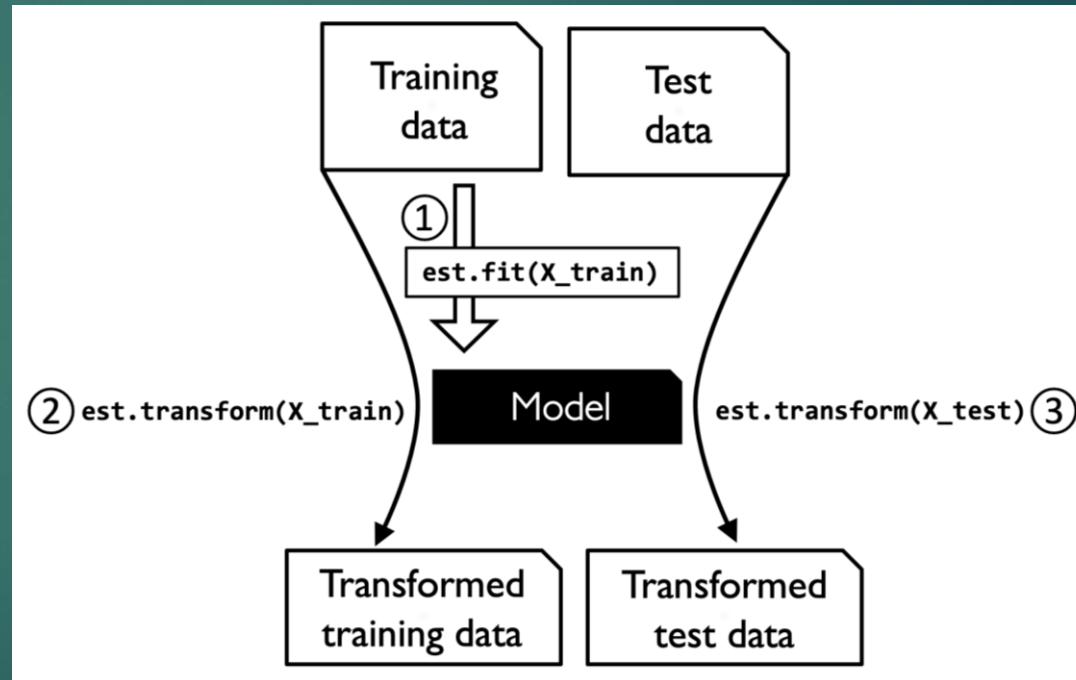
■ Verification of stratification

```
>>> print('Labels counts in y:', np.bincount(y))
Labels counts in y: [50 50 50]
>>> print('Labels counts in y_train:', np.bincount(y_train))
Labels counts in y_train: [35 35 35]
>>> print('Labels counts in y_test:', np.bincount(y_test))
Labels counts in y_test: [15 15 15]
```

Transformer API in Scikit-learn

■ For implementing classes for data transformation

- ▶ `fit()`
- ▶ `transform()`
- ▶ `fit_transform()`



Any data array to be transformed needs to have the same number of features as the data array that was used to fit the model.

sklearn.impute

	A	B	C	D
0	1.0	2.0	3.0	4.0
1	5.0	6.0	NaN	8.0
2	10.0	11.0	12.0	NaN

■ Transformers for missing value imputation

▶ Example: **SimpleImputer** class

- Replace missing values using a descriptive *statistic* (e.g. mean, median, constant or most_frequent) *along each column*

```
>>> from sklearn.impute import SimpleImputer
>>> import numpy as np
➔ >>> imr = SimpleImputer(missing_values=np.nan, strategy='mean')
>>> imr = imr①.fit(df.values)
>>> imputed_data = imr②.transform(df.values)
>>> imputed_data
array([[ 1.,  2.,  3.,  4.],
       [ 5.,  6.,  7.5,  8.],
       [10., 11., 12.,  6.]])
```

① **fit**: Learn the parameters from the training data

② **transform**: Uses those parameters to transform the data

sklearn.preprocessing

- Methods for scaling, centering, normalization, binarization, etc.

► Example: **StandardScaler** class

- Standardize features by removing the mean and scaling to unit variance (standardization)

```
from sklearn.preprocessing import StandardScaler
```

→ `sc = StandardScaler()`

`sc.①fit(X_train)`

`X_train_std = sc.②transform(X_train)`

`X_test_std = sc.②transform(X_test)`

① **fit**: Estimate the parameters μ and σ for each feature from the training data

② **transform**: Standardize the training and testing data using those (same) estimated parameters μ and σ

sklearn.preprocessing

► Example: **MinMaxScaler** class

- Transform features by scaling each feature to a given range, (e.g. between zero and one) (normalization)

```
>>> from sklearn.preprocessing import MinMaxScaler
➔ >>> mms = MinMaxScaler()
>>> X_train_norm = mms.① + ②fit_transform(X_train)
>>> X_test_norm = mms.②transform(X_test)
```

fit_transform method is just a shortcut
for calling fit and transform separately.

sklearn.preprocessing

	color	size	price	classlabel
0	green	1	10.1	class2
1	red	2	13.5	class1
2	blue	3	15.3	class2

► Example: **LabelEncoder** class

- Encode target labels with value between 0 and n_classes-1

```
>>> from sklearn.preprocessing import LabelEncoder
➔ >>> class_le = LabelEncoder()
>>> y = class_le.fit_transform(df['classlabel'].values)
>>> y
array([1, 0, 1])
```

① + ②

```
>>> class_le.inverse_transform(y)
array(['class2', 'class1', 'class2'], dtype=object)
```

sklearn.preprocessing

	color	size	price	classlabel
0	green	1	10.1	1
1	red	2	13.5	0
2	blue	3	15.3	1

Nominal

► Example: OneHotEncoder class

- Encode categorical features as a one-hot numeric array

```
>>> from sklearn.preprocessing import OneHotEncoder
>>> X = df[['color', 'size', 'price']].values
➔ >>> color_ohe = OneHotEncoder()
>>> color_ohe.fit_transform(X[:, 0].reshape(-1, 1)).toarray()
array([[0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.]])
```

Apply the OneHotEncoder to only a single column (X[:, 0].reshape(-1, 1))

- To drop a redundant column (see p. 21)

```
>>> color_ohe = OneHotEncoder(categories='auto', drop='first')
```

sklearn.preprocessing

► Example: **FunctionTransformer** class

- Constructs a transformer from an *arbitrary* callable (e.g., `np.abs`, `np.log`, etc.)
- Useful for stateless transformations such as taking the log, doing custom scaling

```
>>> import numpy as np
>>> from sklearn.preprocessing import FunctionTransformer
→ >>> transformer = FunctionTransformer(np.log1p)
>>> X = np.array([[0, 1], [2, 3]])
>>> transformer.transform(X)
array([[0.        , 0.6931],
       [1.0986, 1.3862]])
```

sklearn.compose

	color	size	price	classlabel
0	green	1	10.1	1
1	red	2	13.5	0
2	blue	3	15.3	1

■ Meta-estimators for building composite models with transformers

▶ Example: **ColumnTransformer** class

- *Selectively* transform columns of an array or DataFrame

```
>>> from sklearn.compose import ColumnTransformer
>>> X = df[['color', 'size', 'price']].values
➔ >>> c_transf = ColumnTransformer([
...     ('onehot', OneHotEncoder(), [0]),
...     ('nothing', 'passthrough', [1, 2])
... ])
>>> c_transf.fit_transform(X).astype(float)
```

A list of tuples:
(name, transformer, column(s))
組合不同資料預處理步驟

'drop': drop the columns
'passthrough': untransformed

`make_column_transformer()` function is a shorthand for the `ColumnTransformer` constructor.

sklearn.feature_selection

■ Feature selection algorithms

▶ Example: **SequentialFeatureSelector** class

- Transformer that performs sequential feature selection

```
>>> from sklearn.feature_selection import SequentialFeatureSelector
>>> from sklearn.neighbors import KNeighborsClassifier
>>> from sklearn.datasets import load_iris
>>> X, y = load_iris(return_X_y=True)
>>> knn = KNeighborsClassifier(n_neighbors=3)
→ >>> sfs = SequentialFeatureSelector(knn, n_features_to_select=3)
>>> sfs.fit(X, y) ①
>>> sfs.get_support()
array([ True, False,  True,  True])
>>> sfs.transform(X).shape
(150, 3) ②
```

estimator

direction : {'forward', 'backward'}, default='forward'

sklearn.feature_selection

► Example: **SelectFromModel** class

- Meta-transformer for selecting features based on importance weights

```
>>> from sklearn.feature_selection import SelectFromModel
>>> from sklearn.linear_model import LogisticRegression
>>> X = [[ 0.87, -1.34,  0.31 ],
...      [-2.79, -0.02, -0.85 ],
...      [-1.34, -0.48, -2.55 ],
...      [ 1.92,  1.48,  0.65 ]]
>>> y = [0, 1, 0, 1]
>>> selector = SelectFromModel(estimator=LogisticRegression()).fit(X, y)
>>> selector.estimator_.coef_
array([[ -0.3252,  0.8345,  0.4976]])
>>> selector.threshold_
np.float64(0.55249)
>>> selector.get_support()
array([False,  True, False])
>>> selector.transform(X)
array([[ -1.34],
       [ -0.02],
       [ -0.48],
       [  1.48]])
```

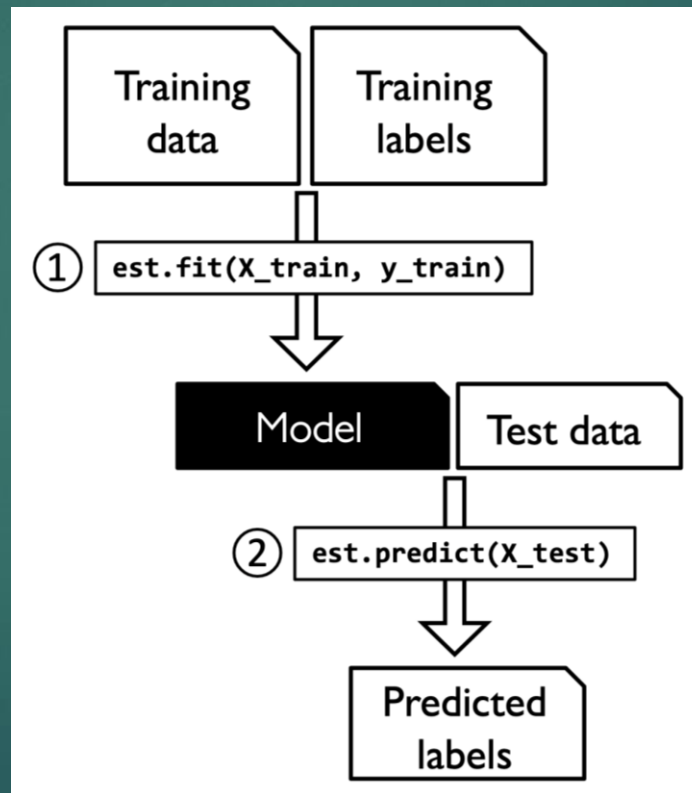
建議設定 L1 regularization
(l1_ratio=1.0, solver='liblinear')

Estimator API in Scikit-learn

- For implementing classes for **estimators** (such as classifiers)

- ▶ **fit()**

- ▶ **predict()**



Train an ML Model with sklearn

- **Initialize an ML object** of specified model/algorithm from sklearn's modules
 - ▶ `sklearn.linear_model`: A variety of linear models
 - ▶ `sklearn.neural_network`: Models based on neural networks (NN)
 - ▶ `sklearn.naive_bayes`: Naive Bayes algorithms
 - ▶ `sklearn.svm`: Support vector machine (SVM) algorithms
 - ▶ `sklearn.neighbors`: The k-nearest neighbors (KNN) algorithms
 - ▶ `sklearn.tree`: Decision tree based models for classification and regression
 - ▶ `sklearn.ensemble`: Ensemble-based methods for classification, regression and anomaly detection
 - ▶ `sklearn.cluster`: Popular unsupervised clustering algorithms
 - ▶ `sklearn.decomposition`: Matrix decomposition (dimensionality reduction) algorithms

Most estimators support L1/L2/elasticnet regularizations via setting the parameters `l1_ratio` and `C`.

Train an ML Model with sklearn

■ Train the model via the `fit` method

- ▶ Most classifiers in scikit-learn *already support* **multiclass classification** by default via the OvA (OvR) method
 - Allows us to feed three flower classes to the classifier all at once
- ▶ Example: `sklearn.linear_model.Perceptron` class
 - Linear perceptron classifier

```
from sklearn.linear_model import Perceptron  
→ ppn = Perceptron(eta0=0.1, random_state=1)  
→ ppn.fit(X_train_std, y_train)
```

Learning rate

Ensure reproducibility of the initial shuffling of the training dataset after each epoch

Train an ML Model with sklearn

- Make predictions via the `predict` method

▶ Example:

```
➔ >>> y_pred = ppn.predict(X_test_std)
>>> print('Misclassified examples: %d' % (y_test != y_pred).sum())
Misclassified examples: 1
```

True class labels

Predicted class labels

- Access the bias unit and weight parameters

▶ `intercept_` (e.g., `ppn.intercept_`)

▶ `coef_` (e.g., `ppn.coef_`)

sklearn.metrics

- Score functions, performance metrics, pairwise metrics and distance computations

► Example: `accuracy_score()` function

- Accuracy classification score

```
>>> from sklearn.metrics import accuracy_score
➔ >>> print('Accuracy: %.3f' % accuracy_score(y_test, y_pred))
Accuracy: 0.978
```

$44/45 \cong 0.978$

True class labels

Predicted class labels

- Alternatively, each classifier in scikit-learn has a `score` method

```
>>> print('Accuracy: %.3f' % ppn.score(X_test_std, y_test))
Accuracy: 0.978
```

$\equiv$ `.predict + accuracy_score()`

Reminder

- You must strictly follow this order:
 - ▶ Split the data using `train_test_split`
 - ▶ Training
 - Fit the transformers (e.g., `SimpleImputer`, `StandardScaler`, `LabelEncoder`, `OneHotEncoder`) only on the training set (`X_train`)
 - Transform the training set (`X_train`)
 - Fit the estimators only on the training set (`X_train`, `y_train`)
 - ▶ Test
 - Transform the test set (`X_test`) using those fitted transformers
 - Predict on the test set (`X_test`) using those fitted estimators

Never use `.fit()` or `.fit_transform()` on the *test* data.

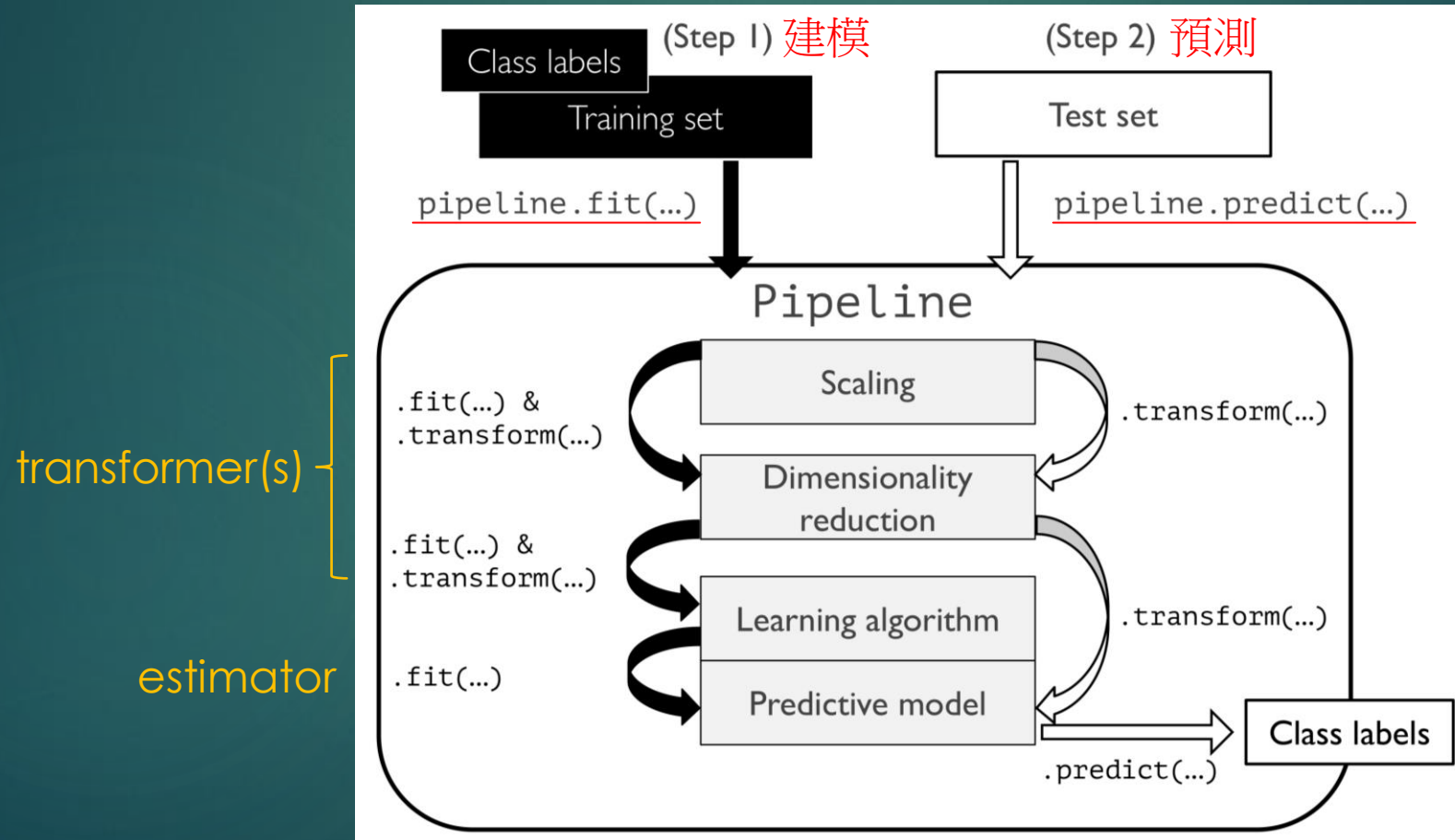
Pipeline

- A meta-estimator (or wrapper) for streamlining workflows
 - ▶ Fit a model including an arbitrary number of transformation steps
 - ▶ Apply it to make predictions about new data

將多個步驟 (例如標準化、特徵選擇、分類器) 組合成一個物件 (一條「自動化產線」)，依序經過清理、轉換、建模，最後可將此物件用於預測結果。這樣的設計讓整個流程自動化、標準化，避免手動分開執行，減少錯誤與重複程式碼，也更容易維護。

Pipeline

■ Inner workings of the Pipeline object



sklearn.pipeline

- Utilities to build a composite estimator as a *chain* of transforms and estimators

- ▶ Example: **Pipeline** class

- A sequence of data transformers with an optional final predictor

```
>>> from sklearn.svm import SVC
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.datasets import make_classification
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.pipeline import Pipeline
>>> X, y = make_classification(random_state=0)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y,
...                                                    random_state=0)
>>> pipe = Pipeline([('scaler', StandardScaler()), ('svc', SVC())])
>>> # The pipeline can be used as any other estimator
>>> # and avoids leaking the test set into the train set
>>> pipe.fit(X_train, y_train).score(X_test, y_test)
0.88
>>> # An estimator's parameter can be set using '__' syntax
>>> pipe.set_params(svc__C=10).fit(X_train, y_train).score(X_test, y_test)
0.76
```

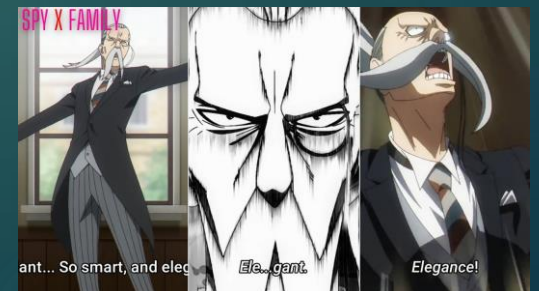
A list of tuples:
(name, transformer/estimator)

sklearn.pipeline

- ▶ Example: `make_pipeline()` function
 - A shorthand for the Pipeline constructor

```
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.decomposition import PCA
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.pipeline import make_pipeline
→ >>> pipe_lr = make_pipeline(StandardScaler(),
...                           PCA(n_components=2),
...                           LogisticRegression())
>>> pipe_lr.fit(X_train, y_train)
>>> y_pred = pipe_lr.predict(X_test)
```

} 2 transformers
estimator





Training a Classifier with Scikit-learn

Mission

- Train a Classifier with Scikit-learn
 - ▶ Build a Pipeline
 - Preprocessing transformers
 - Data cleaning, label encoding, feature scaling and feature selection
 - Logistic regression estimator
- Apply it to the Breast Cancer Wisconsin dataset (乳腺癌威斯康辛資料集)
- Sample code: 05_1.py

Breast Cancer Wisconsin dataset

- 真實醫學資料集 (後製少許遺失值) : [wdbc_missing.csv](#)
 - ▶ 569 筆腫瘤細胞的樣本
 - 212 (M)
 - 357 (B)
 - ▶ 30 種特徵 (從顯微鏡影像中提取)
 - 正實數 或 NaN (缺失值)
 - 例如細胞核大小、形狀、紋理等的平均值/標準誤差/極端值
 - ▶ 二元標籤 (字串型別)
 - M: 惡性 (malignant, /mə'liɡ.nənt/)
 - B: 良性 (benign, /bɪ'naɪn/)

Breast Cancer Wisconsin dataset

■ Some samples



	mean radius	mean texture	mean perimeter	mean area	...	worst concave points	worst symmetry	worst fractal dimension	target
0	17.99	10.38	NaN	1001.0	...	0.2654	0.4601	0.11890	M
1	NaN	17.77	132.90	1326.0	...	0.1860	0.2750	0.08902	M
2	19.69	21.25	130.00	1203.0	...	0.2430	0.3613	0.08758	M
3	11.42	20.38	77.58	386.1	...	0.2575	0.6638	0.17300	M
4	20.29	14.34	135.10	1297.0	...	0.1625	0.2364	0.07678	M
..	...	...	...	...	...	...	...	...	...
564	21.56	22.39	142.00	1479.0	...	0.2216	0.2060	0.07115	M
565	20.13	28.25	131.20	1261.0	...	0.1628	0.2572	0.06637	M
566	16.60	28.08	108.30	858.1	...	0.1418	0.2218	0.07820	M
567	20.60	29.33	140.10	1265.0	...	0.2650	0.4087	0.12400	M
568	7.76	24.54	47.92	181.0	...	0.0000	0.2871	0.07039	B

[569 rows x 31 columns]

30 features

► Preprocessing requirement

- Data cleaning, label encoding, feature scaling and feature selection

Train a Classifier with Scikit-learn

```
1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3  from sklearn.impute import SimpleImputer
4  from sklearn.preprocessing import LabelEncoder, StandardScaler
5  from sklearn.feature_selection import SelectFromModel
6  from sklearn.linear_model import LogisticRegression
7  from sklearn.pipeline import Pipeline
8  from sklearn.metrics import accuracy_score, classification_report
9
10 # 1. 讀取資料集 raw data
11 df_text = pd.read_csv("wdbc_missing.csv")
12
13 # 分離原始特徵 X 與 字串標籤 y
14 X = df_text.drop(columns=['target']).to_numpy()
15 y_raw = df_text['target'].to_numpy()
16
17 # 2. 使用 LabelEncoder 將字串標籤 y 轉成 0 和 1 的整數
18 label_encoder = LabelEncoder()
19 y_encoded = label_encoder.fit_transform(y_raw)
20
21 # 查看轉換後的對應關係 (確認依據字母順序排序, B -> 0, M -> 1)
22 print("\n--- LabelEncoder 轉換對照 ---")
23 for index, class_name in enumerate(label_encoder.classes_):
24     print(f"字串 '{class_name}' 被編碼為整數: {index}")
```

To be covered in Lecture 6

```
--- LabelEncoder 轉換對照 ---
字串 'B' 被編碼為整數: 0
字串 'M' 被編碼為整數: 1
```

Train a Classifier with Scikit-learn

```
26 # 3. 黃金法則：進行任何 Processing 之前，先切分資料集
27 X_train, X_test, y_train, y_test = train_test_split(
28     X, y_encoded, test_size=0.2, random_state=42
29 )
30
31 # 4. 建立特徵處理的管線 (Pipeline 只對接特徵矩陣 X)
32 pipeline = Pipeline([
33     ('imputer', SimpleImputer(strategy='mean')),
34     ('scaler', StandardScaler()),
35     ('feature_selection', SelectFromModel(
36         LogisticRegression(l1_ratio=1.0, solver='liblinear', C=0.1, random_state=42)
37     )),
38     ('classifier', LogisticRegression(l1_ratio=0.0, C=100, max_iter=1000, random_state=42))
39 ])
40
41 # 5. 訓練模型
42 pipeline.fit(X_train, y_train)
43
44 # 查看特徵選取後的結果
45 selected_features_mask = pipeline.named_steps['feature_selection'].get_support()
46 n_features_selected = selected_features_mask.sum()
47 print(f"特徵選取後剩餘數量: {n_features_selected} \
48 (淘汰了 {X.shape[1] - n_features_selected} 個稀疏或高共線性的雜訊特徵)")
```

Regularization strength $\propto \frac{1}{C}$

$\equiv L1$

$\equiv L2$

特徵選取後剩餘數量: 11 (淘汰了 19 個稀疏或高共線性的雜訊特徵)

Train a Classifier with Scikit-learn

```
51 # 6. 在測試集上預測
52 y_pred = pipeline.predict(X_test)
53
54 # 7. 最終評估結案報告
55 print("\n=== 模型最終評估報告 ===")
56 print("測試集最終正確率 (Test Accuracy):", accuracy_score(y_test, y_pred))
57
58 # 把 0, 1 還原回原來的字串 B, M, 印出測試報告
59 print("\n詳細分類報告 (Classification Report):")
60 print(classification_report(y_test, y_pred, target_names=label_encoder.classes_))
```

```
=== 模型最終評估報告 ===
測試集最終正確率 (Test Accuracy): 0.9736842105263158

詳細分類報告 (Classification Report):
```

	precision	recall	f1-score	support
B	0.99	0.97	0.98	71
→ M	0.95↓	0.98↑	0.97	43
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

→ To be covered in Lecture 6

Summary

■ 本範例完成了一個完整的端到端 (End-to-End) 機器學習專案原型

- ▶ 資料切分：train_test_split 切分獨立的訓練集與測試集
- ▶ 資料清理：SimpleImputer 補齊缺失值
- ▶ 資料編碼：LabelEncoder 轉譯文字標籤
- ▶ 特徵縮放：StandardScaler 標準化
- ▶ 特徵選取：SelectFromModel(L1) 移除雜訊特徵
- ▶ 模型訓練：LogisticRegression 分類器
- ▶ 模型評估：classification_report 分類效能指標

To be covered in Lecture 6

■ 只欠缺高規格的模型驗證與超參數調整 (ML 的痛點 🤔)

鎮定點! 我們有 GridSearchCV 代勞~
(Cross Validation + Grid Search)



Thank you